

ImageSense

Documentation technique et modèles mathématiques

Projet d'analyse colorimétrique d'images

13 août 2026

Table des matières

Résumé	3
Abstract	3
1 Introduction	4
2 Contexte et objectifs	4
2.1 Objectifs fonctionnels	4
2.2 Contraintes techniques	4
3 Architecture logicielle	4
3.1 Vue d'ensemble	5
3.2 Description des modules	5
3.3 Cycle de vie de l'application	6
4 Modèles mathématiques	6
4.1 Espace colorimétrique CIELAB	6
4.2 Conversion RGB $\rightarrow$ LAB	6
4.3 Luminance perceptuelle (Rec. 601)	7
4.4 Échantillonnage aléatoire des pixels	7
4.5 Clustering KMeans	7
4.6 Choix automatique de K — score de silhouette	8
4.7 Calcul des proportions	8
4.8 Nommage des couleurs — distance ΔE	9
4.9 Histogramme de luminance et statistiques tonales	9
4.10 Modèles de retouche d'image	9
5 Pipeline d'analyse	11
6 Interface utilisateur	11
7 Exports de données	12
8 Complexité et performances	12
9 Limites connues	12
10 Conclusion	13

Résumé

ImageSense est une application web d'analyse colorimétrique et de retouche d'image en temps réel. L'utilisateur téléverse une image (JPG, PNG, WEBP), la retouche via une barre latérale inspirée de Photoshop (luminosité, contraste, saturation, gamma, exposition, température, vibrance, vignettage, filtres artistiques...), puis lance une analyse qui extrait automatiquement les **couleurs dominantes**, estime leurs **proportions** et les classe par **clustering KMeans** dans l'espace perceptuel **CIELAB**.

Le nombre de couleurs est soit choisi manuellement, soit déterminé automatiquement par optimisation du **score de silhouette** sur un intervalle $K \in [2, 8]$. Chaque couleur extraite est nommée approximativement (bleu, vert, gris...) par recherche du plus proche voisin dans un dictionnaire de 67 couleurs de référence, mesurée par **distance euclidienne dans LAB** (ΔE).

Le pipeline repose sur : conversion **RGB** $\rightarrow$ **LAB**, échantillonnage aléatoire ($\leq 30\,000$ pixels), entraînement **KMeans** (10 initialisations), calcul des proportions par comptage de clusters, reconstruction d'une **image segmentée**, puis rendu interactif (palette, graphique de proportions, tableau détaillé) et **export JSON / CSV**. Une fonctionnalité de retouche intégrée affiche les **histogrammes de luminance avant/après** (ombres, tons moyens, hautes lumières) avec statistiques comparatives, mis à jour en direct.

Mots-clés : vision par ordinateur, couleurs dominantes, CIELAB, KMeans, score de silhouette, histogramme, retouche d'image, Streamlit.

Abstract

ImageSense is a web application for colorimetric image analysis and real-time photo editing. The user uploads an image (JPG, PNG, WEBP), edits it through a Photoshop-inspired sidebar (brightness, contrast, saturation, gamma, exposure, temperature, vibrance, vignetting, artistic filters...), then triggers an analysis that automatically extracts **dominant colors**, estimates their **proportions**, and classifies them via **KMeans clustering** in the perceptually-uniform **CIELAB** color space.

The number of colors is either set manually or selected automatically by maximizing the **silhouette score** over $K \in [2, 8]$. Each extracted color is roughly named (blue, green, gray...) by nearest-neighbor search against a dictionary of 67 reference colors, using **Euclidean distance in LAB space** (ΔE).

The pipeline relies on : **RGB** $\rightarrow$ **LAB** conversion, random pixel sampling ($\leq 30,000$ pixels), **KMeans** training (10 initializations), proportion computation by cluster counting, reconstruction of a **segmented image**, then interactive rendering (palette, proportion chart, detailed table) and **JSON / CSV export**. An integrated editing module displays **before/after luminance histograms** (shadows, midtones, highlights) with comparative statistics, updated live.

Keywords : computer vision, dominant colors, CIELAB, KMeans, silhouette score, histogram, image editing, Streamlit.

1. Introduction

L'analyse des couleurs d'une image est un problème classique de vision par ordinateur, utilisé dans les domaines du design, du marketing, de la photographie et du e-commerce. L'objectif est de réduire une image (des centaines de milliers de pixels) à un petit ensemble de couleurs représentatives accompagnées de leurs proportions.

La difficulté principale est que l'espace **RGB** n'est pas **perceptuellement uniforme** : une distance euclidienne dans RGB ne reflète pas la distance perçue par l'œil humain. ImageSense répond à ce problème en travaillant dans l'espace **CIELAB**, conçu pour que la distance euclidienne y approxime la différence perceptuelle, et en utilisant **KMeans** pour regrouper les pixels en clusters de couleurs dominantes.

Ce document présente l'architecture du projet, puis détaille les modèles mathématiques sous-jacents : conversions colorimétriques, clustering, métriques de qualité et modèles de retouche.

2. Contexte et objectifs

Le projet part d'un constat : extraire les couleurs dominantes d'une image de façon fiable nécessite un espace colorimétrique perceptuel et une méthode de clustering robuste. Le score de silhouette permet en outre d'automatiser le choix du nombre de couleurs.

2.1. Objectifs fonctionnels

- Téléverser une image (JPG, JPEG, PNG, WEBP) ;
- Retoucher l'image en temps réel et visualiser l'histogramme de luminance avant / après ;
- Extraire automatiquement les couleurs dominantes ;
- Estimer la proportion de chaque couleur (pixels et pourcentage) ;
- Classifier les couleurs par clustering (KMeans) ;
- Choisir le nombre de couleurs automatiquement (silhouette) ou manuellement ;
- Nommer approximativement chaque couleur (rouge, bleu, gris, etc.) ;
- Générer une image segmentée par clusters ;
- Exporter les résultats en JSON et CSV.

2.2. Contraintes techniques

- Limiter le temps de calcul par redimensionnement (taille max 1 000 px) et échantillonnage ($\leq 30\,000$ pixels) ;
- Assurer la reproductibilité via une graine aléatoire fixe (`RANDOM_STATE = 42`) ;
- Réaliser tous les traitements avec NumPy / scikit-learn / scikit-image.

3. Architecture logicielle

3.1. Vue d'ensemble

L'application suit une architecture modulaire à **injection de dépendances** : le module central `ColorAnalyzer` orchestre des services indépendants, ce qui facilite les tests et la maintenance.

```
ImageSense/
|
|-- app.py # Point d'entree Streamlit (orchestration UI)
|-- constants.py # Constantes : couleurs de reference, bornes K, seuils
|-- models.py # Dataclasses ColorResult, AnalysisMetadata
|-- image_processor.py # Chargement, redimensionnement, echantillonnage
|-- image_editor.py # Retouche facon Photoshop (27 reglages)
|-- histogram_processor.py # Histogrammes luminance/canaux, statistiques tonales
|-- color_processor.py # Conversions RGB->LAB, RGB->HEX, nommage des couleurs
|-- clustering_service.py # KMeans, choix automatique de K (silhouette)
|-- color_analyzer.py # Pipeline d'analyse complet (orchestrateur)
|-- result_renderer.py # Rendu Streamlit : sidebar, images, palette, tableau
|-- styles.py # CSS global + CSS iframe
|-- .streamlit/config.toml # Theme ocean Streamlit
|-- requirements.txt # Dependances Python
+-- DOCUMENTATION.tex # Ce document
```

3.2. Description des modules

Module	Responsabilité
<code>app.py</code>	Configure la page, injecte le CSS, gère l'upload, les onglets Retouche/Analyse et le cycle de session.
<code>constants.py</code>	REFERENCE_COLORS (67 couleurs), formats acceptés, teintes, filtres, bornes $K_{\min} = 2$, $K_{\max} = 8$, $K_{\text{manuel, max}} = 12$, MAX_SAMPLES=30 000, RANDOM_STATE=42.
<code>models.py</code>	ColorResult (rang, nom, rgb, hex, pixels, proportion, ΔE) et AnalysisMetadata (dimensions, mode, K, silhouette).
<code>ImageProcessor</code>	<code>load_image</code> (PIL $\rightarrow$ RGB, thumbnail), <code>sample_pixels</code> (tirage sans remise), <code>rebuild_image_from_labels</code> (mapping labels $\rightarrow$ couleurs).
<code>ImageEditor</code>	Applique dans un ordre déterminé 27 réglages : noir & blanc, sépia, filtres artistiques, canaux, température, teinte, vibrance, saturation, ombres/hautes lumières, exposition, gamma, luminosité, contraste, netteté, flou gaussien, grain, postérisation, solarisation, seuil, vignette, pixelisation, teinte colorée, négatif.
<code>HistogramProcessor</code>	Luminance perceptuelle, histogramme luminance (64 bins) et canaux, comparaison avant/après, statistiques tonales.
<code>ColorProcessor</code>	<code>rgb2lab</code> / <code>lab2rgb</code> (scikit-image), <code>rgb_to_hex</code> , <code>get_color_name</code> (plus proche voisin LAB).
<code>ClusteringService</code>	<code>choose_best_k</code> (silhouette sur $K \in [2, 8]$), <code>fit</code> (KMeans, <code>n_init</code> =10), <code>predict</code> .
<code>ColorAnalyzer</code>	Enchaîne les étapes du pipeline et construit les objets de résultat.
<code>ResultRenderer</code>	Rendu Streamlit : sidebar, hero, cartes de métriques, palette avec copie HEX, graphique Altair, tableau, exports.

3.3. Cycle de vie de l'application

1. **Session initiale** — pas d'image : écran vide avec consigne de téléversement.
2. **Upload** — les octets sont conservés dans `st.session_state["uploaded_bytes"]` pour éviter un re-téléversement.
3. **Retouche en temps réel** — chaque réglage de la sidebar déclenche `ImageEditor.apply()` et régénère les histogrammes avant/après.
4. **Analyse** — au clic sur le bouton, `ColorAnalyzer.analyze()` est exécuté ; les résultats sont stockés dans `st.session_state`.
5. **Rendu** — palette, graphique des proportions, tableau, exports JSON/CSV.

4. Modèles mathématiques

4.1. Espace colorimétrique CIELAB

CIELAB est un espace perceptuellement quasi-uniforme : une différence de valeur y correspond environ à la même différence perçue, quelle que soit la région de l'espace.

- L^* : clarté, $L^* \in [0, 100]$ (noir $\rightarrow$ blanc) ;
- a^* : axe vert $\rightarrow$ rouge, $a^* \in [-128, 127]$;
- b^* : axe bleu $\rightarrow$ jaune, $b^* \in [-128, 127]$.

La distance euclidienne dans LAB définit la **différence de couleur** ΔE :

$$\Delta E = \sqrt{(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2} \quad (1)$$

C'est cette propriété qui justifie le choix de LAB pour le clustering et le nommage des couleurs.

4.2. Conversion RGB $\rightarrow$ LAB

La conversion passe par l'espace intermédiaire **CIE XYZ**.

Étape 1 — linéarisation sRGB. Pour chaque canal $c \in [0, 1]$:

$$c' = \begin{cases} \frac{c}{12.92} & \text{si } c \leq 0.04045 \\ \left(\frac{c + 0.055}{1.055} \right)^{2.4} & \text{sinon} \end{cases} \quad (2)$$

Étape 2 — passage sRGB linéaire $\rightarrow$ XYZ. Matrice du blanc D65 :

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix} \begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} \quad (3)$$

Étape 3 — normalisation au blanc de référence. Avec $X_n = 0.95047$, $Y_n = 1.0$, $Z_n = 1.08883$, on définit la fonction de transfert :

$$f(t) = \begin{cases} t^{1/3} & \text{si } t > \left(\frac{6}{29}\right)^3 \\ \frac{1}{3} \left(\frac{29}{6}\right)^2 t + \frac{4}{29} & \text{sinon} \end{cases} \quad (4)$$

Étape 4 — obtention de $L^*a^*b^*$.

$$L^* = 116 f\left(\frac{Y}{Y_n}\right) - 16 \quad (5)$$

$$a^* = 500 \left[f\left(\frac{X}{X_n}\right) - f\left(\frac{Y}{Y_n}\right) \right] \quad (6)$$

$$b^* = 200 \left[f\left(\frac{Y}{Y_n}\right) - f\left(\frac{Z}{Z_n}\right) \right] \quad (7)$$

La conversion inverse **LAB** $\rightarrow$ **RGB** (utilisée pour les centres de clusters) applique les transformations inverses, le tout implémenté par scikit-image (`rgb2lab`, `lab2rgb`).

4.3. Luminance perceptuelle (Rec. 601)

La luminance de chaque pixel pondère les canaux selon la sensibilité de l'œil (le vert est dominant) :

$$Y = 0.299 R + 0.587 G + 0.114 B, \quad Y \in [0, 255] \quad (8)$$

Cette formule est utilisée par `HistogramProcessor` et par la retouche (ombres, hautes lumières, seuil).

4.4. Échantillonnage aléatoire des pixels

Pour accélérer l'entraînement de KMeans, un sous-ensemble de $S = 30\,000$ pixels est tiré **sans remise** selon une loi uniforme, à graine fixe :

$$\mathcal{S} = \{X_{i_j}\}_{j=1}^S, \quad i_j \sim \mathcal{U}\{1, N\} \text{ sans remise} \quad (9)$$

avec $N = H \times W$ le nombre total de pixels. Le clustering est appris sur $\mathcal{S}$ puis appliqué à la totalité des pixels (prédiction).

4.5. Clustering KMeans

Objectif. Partitionner les N pixels $\{x_1, \dots, x_N\} \subset \mathbb{R}^3$ (coordonnées LAB) en K clusters $C_1, \dots, C_K$ de centres $\mu_1, \dots, \mu_K$, en minimisant l'inertie intra-cluster (somme des carrés intra-cluster, WCSS) :

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|_2^2 \quad (10)$$

Algorithme. Alternance de deux étapes jusqu'à convergence, avec `n_init` = 10 initialisations (la meilleure inertie est conservée) :

1. **Initialisation** k-means++ (défaut scikit-learn), graine fixe = 42 ;
2. **Affectation** : chaque pixel est assigné au centre le plus proche :

$$C_i^{(t)} = \left\{ x : \|x - \mu_i^{(t)}\|_2 = \min_j \|x - \mu_j^{(t)}\|_2 \right\} \quad (11)$$

3. **Mise à jour** : chaque centre devient le barycentre de son cluster :

$$\mu_i^{(t+1)} = \frac{1}{|C_i^{(t)}|} \sum_{x \in C_i^{(t)}} x \quad (12)$$

4.6. Choix automatique de K — score de silhouette

Le **score de silhouette** mesure la cohésion d'un pixel avec son propre cluster (distance intra a_i) relativement à la distance au cluster voisin le plus proche (b_i) :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad s(i) \in [-1, 1] \quad (13)$$

- $s(i) \approx 1$: pixel très bien classé ;
- $s(i) \approx 0$: pixel à la frontière de deux clusters ;
- $s(i) < 0$: pixel probablement mal affecté.

Le score global est la moyenne sur tous les pixels :

$$\bar{s} = \frac{1}{N} \sum_{i=1}^N s(i) \quad (14)$$

Stratégie. Pour chaque $K \in [2, 8]$, entraîner KMeans, calculer $\bar{s}$ sur un échantillon de 5 000 pixels, et retenir :

$$K^* = \arg \max_K \bar{s}(K) \quad (15)$$

4.7. Calcul des proportions

Après prédiction, le nombre de pixels de chaque cluster n_j est compté (`np.bincount`), et la proportion est :

$$p_j = \frac{n_j}{\sum_{k=1}^K n_k}, \quad \sum_j p_j = 1 \quad (16)$$

Les couleurs sont ensuite classées par proportion décroissante (rang 1 = couleur la plus présente).

4.8. Nommage des couleurs — distance ΔE

Chaque centre de cluster $L_c = (L^*, a^*, b^*)$ est comparé aux 67 couleurs de référence R_k (converties au préalable en LAB) par distance euclidienne :

$$k^* = \arg \min_k \|L_c - R_k\|_2 \quad (17)$$

Le nom retenu est celui de la couleur de référence la plus proche, et la valeur $\|L_c - R_{k^*}\|_2$ est stockée dans `distance_lab` comme indicateur de confiance.

4.9. Histogramme de luminance et statistiques tonales

Histogramme. L'intervalle $[0, 255]$ est découpé en $B = 64$ tranches de largeur $\Delta = 255/64$. Pour la tranche j de bornes $[e_j, e_{j+1}]$:

$$h_j = \sum_x \mathbb{K}\{e_j \leq Y(x) < e_{j+1}\} \quad (18)$$

Zones tonales. Seuils : ombres ≤ 85 , hautes lumières ≥ 170 .

- Ombres : $[0, 85)$;
- Tons moyens : $[85, 170)$;
- Hautes lumières : $[170, 255]$.

Le pourcentage de pixels par zone est :

$$\%_{\text{zone}} = 100 \times \mathbb{E}[\mathbb{K}\{Y \in \text{zone}\}] \quad (19)$$

Statistiques. Moyenne $\mu = \frac{1}{N} \sum_i Y_i$, médiane, écart-type σ (indicateur de contraste), min, max — calculés sur la luminance et sur chaque canal RGB.

4.10. Modèles de retouche d'image

Les opérations élémentaires de `ImageEditor` sont les suivantes.

Exposition (EV). Chaque +1 EV double la luminosité :

$$I' = I \cdot 2^{EV} \quad (20)$$

Gamma. Correction de courbe :

$$I' = 255 \left(\frac{I}{255} \right)^{1/\gamma} \quad (21)$$

Sépia. Combinaison linéaire des canaux :

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} 0.393 & 0.769 & 0.189 \\ 0.349 & 0.686 & 0.168 \\ 0.272 & 0.534 & 0.131 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (22)$$

Température. Décalage additif (chaud : $+R$, $-B$; froid : inverse) :

$$R' = R + \frac{t}{100} \cdot 45, \quad G' = G + \frac{t}{100} \cdot 12, \quad B' = B - \frac{t}{100} \cdot 45 \quad (23)$$

Canaux (gains).

$$I'_c = I_c + g_c \cdot 1.275 \quad (24)$$

Vibrance (espace HSV). Renforce surtout les couleurs ternes, avec $v = \text{vibrance}/100$:

$$S' = \begin{cases} S + v(255 - S) \cdot 0.7 & \text{si } v \geq 0 \\ S(1 + v) & \text{si } v < 0 \end{cases} \quad (25)$$

Rotation de teinte.

$$H' = (H + \text{offset}) \bmod 256 \quad (26)$$

Ombres. Pondération forte sur les zones sombres, $w = (1 - Y/255)^2$:

$$I' = I + \frac{s}{100} \cdot w \cdot 120 \quad (27)$$

Hautes lumières. Pondération forte sur les zones claires, $w = (Y/255)^2$:

$$I' = I + \frac{h}{100} \cdot w \cdot 120 \quad (28)$$

Vignette. Facteur radial décroissant depuis le centre (d distance normalisée, $d \in [0, 1]$) :

$$d = \sqrt{\left(\frac{x - W/2}{W}\right)^2 + \left(\frac{y - H/2}{H}\right)^2} / 0.7071 \quad (29)$$

$$f = 1 - \frac{v}{100} d^2, \quad I' = I \cdot f \quad (30)$$

Postérisation. Quantification à L niveaux par canal :

$$I' = \frac{\text{round}(I \cdot \frac{L-1}{255})}{(L-1)/255} \quad (31)$$

Solarisation. Effet Sabattier, inversion au-delà d'un seuil t :

$$I' = \begin{cases} I & \text{si } I < t \\ 255 - I & \text{si } I \geq t \end{cases} \quad (32)$$

Seuil (noir & blanc).

$$I' = \begin{cases} 255 & \text{si } Y \geq t \\ 0 & \text{sinon} \end{cases} \quad (33)$$

Grain de film. Bruit gaussien additif, $\sigma = \frac{\text{grain}}{100} \cdot 50$:

$$I' = I + \mathcal{N}(0, \sigma) \quad (34)$$

Teinte colorée. Interpolation entre l'image et une couleur cible C , $t = \frac{\text{intensité}}{100}$:

$$I' = I(1 - t) + C t \quad (35)$$

Négatif.

$$I' = 255 - I \quad (36)$$

Luminosité, contraste, saturation, netteté. Mises à l'échelle linéaire par `ImageEnhance`, facteur = valeur/100.

Pixelisation. Réduction puis agrandissement avec filtre au plus proche voisin.

Toutes les opérations sont suivies d'un `clip(I, 0, 255)` et d'un cast `uint8`.

5. Pipeline d'analyse

Le pipeline complet exécuté par `ColorAnalyzer.analyze()` :

```
Image RGB (H, W, 3)
| 1. Conversion RGB -> LAB
V
Image LAB (H, W, 3)
| 2. Aplatissement -> pixels (H*W, 3)
| 3. Echantillonnage (<= 30 000 px, graine 42)
V
Pixels LAB echantillonnes
| 4. K automatique (silhouette, K in [2,8]) OU K manuel
| 5. KMeans (n_init=10) + prediction sur tous les pixels
V
Labels (H*W,)
| 6. Comptage (bincount) -> proportions
| 7. Centres LAB -> RGB
| 8. Reconstruction image segmentee (H, W, 3)
| 9. Tri par proportion decroissante
| 10. Nommage (Delta E) + HEX + ColorResult
V
Resultats : couleurs, image segmentee, metadonnees
```

6. Interface utilisateur

- **Thème océan** personnalisé (`.streamlit/config.toml`) : sarcelle #137C8B, ardoise #344D59, fond bleuté #E8F1F3;
- **Hero** avec dégradé et badges récapitulatifs;

- **CSS centralisé** (`styles.py`) : cartes de métriques, cadres d'images, palette avec rang, survol, copie des codes HEX au clic ;
- **Barre latérale** organisée en sections repliables : luminosité, tons, détail, effets avancés, filtres artistiques, paramètres d'analyse ;
- **Onglet Retouche** : comparaison avant/après + histogrammes luminance (zones ombres / tons moyens / hautes lumières) et canaux RGB superposés, statistiques comparatives mises à jour en direct ;
- **Onglet Analyse** : image segmentée, palette de couleurs, graphique des proportions coloré (Altair), tableau détaillé, exports JSON/CSV.

7. Exports de données

- **JSON** : liste des `ColorResult` (rang, nom, rgb, hex, pixels, proportion, proportion_pct, distance_lab) + `AnalysisMetadata` ;
- **CSV** : tableau des couleurs avec colonnes renommées et formatées.

8. Complexité et performances

Opération	Complexité	Notes
Échantillonnage	$O(N)$	$N \leq W \times H$, borné à 30 000 px pour l'entraînement
KMeans	$O(K \cdot n \cdot d \cdot \text{iter} \cdot n_{\text{init}})$	$n = 30\,000$, $d = 3$, $n_{\text{init}} = 10$, au plus 7 valeurs de K ($K \in [2, 8]$)
Score silhouette	$O(n^2)$ en théorie	borné par échantillonnage à 5 000 px
Prédiction	$O(N \cdot K \cdot d)$	N = tous les pixels de l'image redimensionnée
Reconstruction	$O(N)$	tableaux indexés NumPy

Le redimensionnement (taille max 600 px par défaut, réglable jusqu'à 1 000 px) garantit un temps de réponse interactif dans Streamlit.

9. Limites connues

- **Nommage approximatif** : le nom dépend du dictionnaire de 67 références ; deux nuances proches peuvent recevoir le même nom, et le seuil de ΔE n'est pas discriminé (une couleur hors-gamme est toujours nommée).
- **KMeans sensible à l'initialisation** : atténué par `n_init` = 10 et la graine fixe ; les clusters sont convexes (forme sphérique), ce qui peut mal séparer des couleurs non linéairement séparables.
- **Choix de K par silhouette** : limité à $[2, 8]$; le score peut favoriser un K faible pour des images très hétérogènes.
- **Précision des conversions** : $\text{LAB} \rightarrow \text{RGB}$ peut produire des valeurs hors gamme, gérées par `clip` (légère perte de fidélité).

- **Redimensionnement** : l’analyse porte sur une image réduite, les pixels fins peuvent être perdus.

10. Conclusion

ImageSense combine des concepts mathématiques éprouvés — espace perceptuel CIELAB, clustering KMeans, score de silhouette, distance ΔE — dans une application web interactive et professionnelle. L’architecture modulaire (injection de dépendances) facilite l’extension : nouveaux espaces colorimétriques (HSV, HSL), clustering hiérarchique ou DBSCAN, seuillage de ΔE pour le nommage, ou analyse par régions constituent les pistes d’évolution naturelles.